

Applications	
CoAP Requests/Responses	
CoAP Messaging	
UDP/DTLS	
IPv4,IPv6	6LoWPAN
Ethernet/WiFi	WPAN(802.15.4)
Physical Channel	

Figure 1: Protocol stack with CoAP support.

(CON) messages, where each message (request/response) should be acknowledged (ACK) by the peer endpoint. Messages will be retransmitted if a CON message is not acknowledged within the time out. A response may be combined with an ongoing acknowledgement, which is known as a piggy backed response, or the response may be sent afterwards if not available immediately, which is known as a separate response. Piggy backed responses need not be acknowledged, and the same holds for non conformable (NON) messages.

Req/Resp code: A 3-bit class ID and 5-bit detail in the *c.dd* format forms this field. The class values used for requests, success responses, client error responses and server error responses are 0, 2, 4 or 5, respectively. The detail carries the request code or response code depending on the class value. Code 0.00 indicates an empty message.

Message ID: A 16-bit unsigned number in network byte order is used to match acknowledgement or reset messages and eliminate duplicate messages.

Tokens: An optional token field, which is limited to 0 to 8 bytes currently to match request responses, may be kept after the header, which is of TKL number of bytes specified in the header.

Options: Zero or more option fields may follow a token. A few options are Content Format, Accept, Max-Age, Etag, Uri-Path, Uri-Query, etc.

CoAP URIs: CoAP URIs consist of the hostname, port number, path and query string, which are specified by option fields Uri-Host, Uri-Port, Uri-Path and Uri-Query, of which Uri-Host and Uri-Port are implicit as they are part of underlying layers. Uri-Path and Uri-Query are significant and part of the CoAP message. For example, if we request a resource with URI *coap://hostname:port/leds/red?q=state&on*, the following options are generated:

```
Option#1 Uri-Path leds,
Option#2 Uri-Path red,
Option#3 Uri-Query q=state
Option#4 Uri-Query on.
```

Ver(2)	Type(2)	Tkl(4)	Req/Resp code(8)	Message ID (16)
Token (0-8) Tk1 byte				
Option if (any)				
Payload Marker (0xff)		Payload (if any)		

Figure 2: CoAP message format

CoAP servers run on UDP port no.5683 by default.

Payload: Followed by the optional token and zero or more option fields, the payload may start, in which case a payload marker 0xFF is placed inbetween. This indicates the end of the options and the start of the payload. The absence of the payload marker indicates an empty payload.

Request methods

CoAP has come up with a subset of the Representational State Transfer [REST] architecture of the Web, which resembles HTTP with request methods GET(1), POST(2), PUT(3) and DELETE(4), with indicated request codes. These methods can be used to create, update, query and delete the resources on the server representing events of an IoT application. For example, a server getting sensor readings directly can define suitable resources and provide data when the GET method is requested by clients, or sensor readings can be updated by using the POST method by a client holding sensor values. Besides, resources can be updated using the GET method with a query string. Similarly, clients can use the POST method to update resources which may represent actuators. However, the PUT method is preferred to POST for idempotent operations.

A *Discover* operation is added with a *GET* request on the predefined path */.well-known/core*, which returns a list of available resources along with applicable paths in the response. A ping operation is implemented by sending an empty message to which the peer responds with an empty reset (RST) message indicating the 'liveness' of the endpoint.

Additionally the *Observe* method is defined, where the notification is sent periodically or on an event basis, for a single request. If the request is a CON message, each response is of the CON type and will be acknowledged by the client. *Observe* can be initiated with an extended GET request, with the added *Observe* option set to zero. It can be cancelled by sending a reset message for one of the notifications or by sending an explicit GET request with the *Observe* option set to value 1. Notifications use the *Observe* option with sequential values for reordering of delayed responses and the *Max-Age* option to keep freshness in the cache. The combination of resource update, *Observe* method is analogous to publish-subscribe mechanisms in other protocols like MQTT. CoAP also offers block wise transfers, which enable a large amount of data exceeding datagram limits, and eliminates fragmentation at